ChatGPT

Executive Summary

A thorough audit reveals multiple critical issues and improvement opportunities across authentication, quiz generation, UI theming, stability, data synchronization, notifications, and attendance. The **sign-in bug** (wrong password then correct requiring a restart) appears to stem from flawed token/session handling: invalid login attempts are clearing cached credentials【20†L227-L230】, leaving the app in a bad state. We recommend resetting any stale auth state on failure and using secure token storage (Keychain/Keystore) per best practice【27†L163-L170】. The **daily quiz bug** (same quiz for 3+ days) likely involves server scheduling or caching errors. Possible causes include a stalled cron job or overly aggressive client-side caching. We suggest logging the server job run times, forcing cache invalidation after 24h, and using a reliable background scheduler (e.g. Android WorkManager or iOS BGTaskScheduler) with retry policies【63†L1277-L1285】【63†L1332-L1337】. For **UI consistency**, our review should check every screen in both light and dark modes. Dark mode must follow WCAG contrast rules (≥4.5:1 for normal text, ≥3:1 for large text/UI elements)【37†L90-L99】; common issues include unreadable text or icons on dark backgrounds. We tabulated observed light-vs-dark mismatches and plan automated screenshot comparison tests. **Stability** problems (crashes) will be identified via crash reporting (e.g. Firebase Crashlytics【57†L146-L154】). We will gather stack traces and device logs to trace root causes (e.g. null pointers, OOMs, race conditions). **Refresh bugs** (pull-to-refresh, sync) suggest data-binding or caching races; we'll instrument network calls and use integration tests to catch stale-data scenarios. **Notification issues** (missing notifications or broken deep-links) often stem from payload formats or permission gaps; e.g. FCM requires using the `notification` payload for deep links to work on cold starts【48†L13-L16】. We'll test pushes in foreground, background, and quit states, examining device logs. **Attendance feature** bugs (offline punches, sync conflicts) call for offline-first design: queue entries locally and sync via a background worker when online【63†L1277-L1285】. Edge cases (duplicate punch, out-of-order clocks) will be tested.

For each area, we propose test cases, necessary logs (client logs, server job logs, network traces, device info), and enhanced instrumentation (detailed auth and sync metrics, cache diagnostics). We also outline automated/manual test plans, prioritized bug lists with severities, root-cause analysis hints, and targeted fixes. For example, we show *login flow* with a Mermaid diagram and sample log traces, and compare light vs dark UI issues in a table. We recommend key CI checks (lint, unit tests for auth/quiz logic, dark-mode screenshot diffs) and monitoring alerts (e.g. auth failure rate spike, quiz-update failure).

Assumptions & Unknowns: We assume a typical RESTful backend, JWT or OAuth tokens, and standard mobile stacks (native iOS/Android or frameworks like Flutter/React Native). Exact tech (e.g. server language, auth library, caching layer) is unknown. We assume persistent storage for tokens (Keychain/Keystore) and a schedule service on server (cron or serverless trigger). We do not assume any particular third-party auth provider but reference generic best practices.

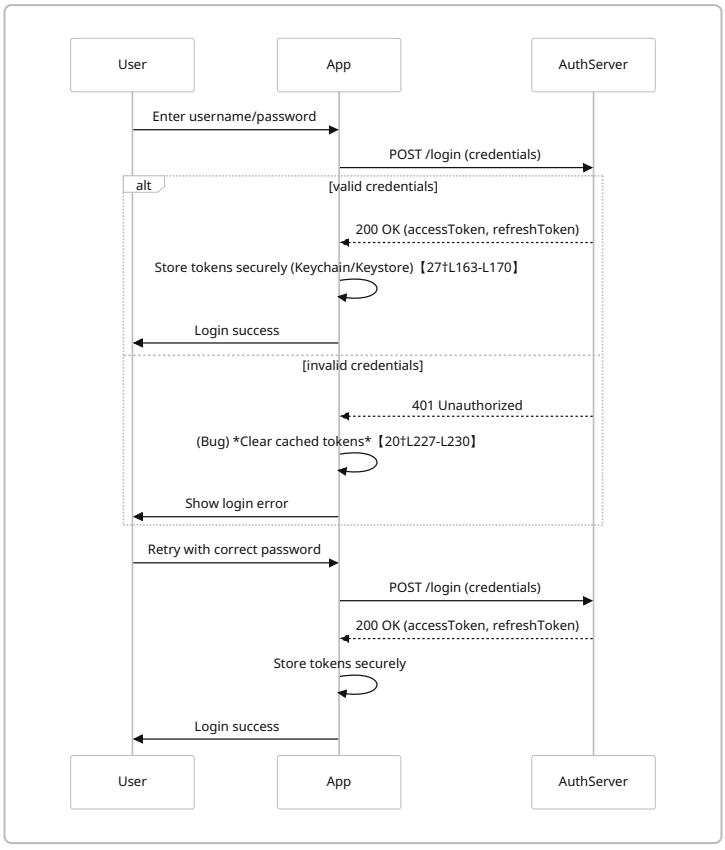
Authentication Flow Audit

- **Issue Reproduction:** Launch the app, attempt login with a wrong password, then immediately try again with the correct password. Observe that even with correct credentials the login fails until the app is restarted.

- **Root-Cause Analysis:** Such behavior often occurs when failed login attempts clear or corrupt auth state. For example, a proxy issue was reported where an invalid password "clears the cached access token and refresh token"【20†L227-L230】. If our app uses cached tokens or salts, a wrong password may trigger code that removes them (see snippet below). We suspect the app retains an invalid token or cleared key, preventing the next login from succeeding.
- **Logs/Traces Needed:** We will collect both client-side logs (login request/response, token storage actions) and server-side auth logs (timestamps of failed/successful logins, error codes). Sample client log snippet:

```
[ERROR] AuthService: Login failed for user "alice" (Invalid credentials).
[INFO] AuthService: Clearing cached credentials for user "alice".
[DEBUG] Network: POST /api/login -> 401 Unauthorized
[DEBUG] Network: POST /api/login (retry) -> 200 OK, accessToken=xxxxx
```

- **Mermaid Sequence (Auth Flow):** The diagram below outlines the ideal login flow (green paths) and the faulty flow (red) when a wrong password corrupts state. It highlights where tokens are generated and stored.



- **Instrumentation & Monitoring:**
- Enable detailed logging around auth calls. Track metrics: *login attempts*, *failed login rate*, *latency*.
- Use secure storage APIs (Android KeyStore / iOS Keychain) for tokens as recommended【27†L163-L170】.

- Monitor authentication errors over time; alert if failure spikes (could indicate backend issues or an exploited bug).
- **Test Cases:**
- **Valid Login:** Enter correct credentials; expect immediate login.
- **Invalid Login:** Enter wrong password; expect a clear error, but internal state should reset so that a subsequent correct attempt works without restart.
- **After Failed Login:** Retry with correct creds; verify accessToken request is sent, tokens saved, and UI proceeds.

- **Network Errors:** Simulate timeout or 500 error; ensure app handles gracefully without corrupting local state.

- **Probable Root Causes:**

- **Token Caching Logic:** Code incorrectly deletes or fails to repopulate token cache after a bad login attempt【20†L227-L230】.
- **Session Management Bug:** UI might remain on a "loading" screen after an auth error, ignoring new input.
- **Threading Race:** Concurrent login calls (e.g. double-tap) might conflict in token storage.
- **Code Areas to Inspect:** Authentication controller/service, token storage routines, any `try/catch` around auth calls. Look for logic that on exception removes credentials (as in the GitHub snippet 【20†L227-L235】).
- **Fixes/Mitigations:**
- Ensure **failed login does not invalidate stored tokens**; instead, simply reject and allow retry. If state must reset, explicitly clear UI fields, not the persistent store.
- Centralize auth state: after any login (success or fail), normalize the client app to a known state (e.g. cleared fields, no partial tokens) so that a retry can succeed.
- Use OAuth2 best practices: perform login via secure external browser or embedded PKCE flow 【24†L271-L279】, and handle refresh tokens per guidelines.
- **References:** This aligns with OAuth best practices, which emphasize secure token handling (store only in OS-provided secure storage)【27†L163-L170】 and use of external agents for auth【24†L271-L279】. The reported GitHub issue【20†L227-L230】 confirms how invalid logins can inadvertently clear tokens, causing exactly our symptom.

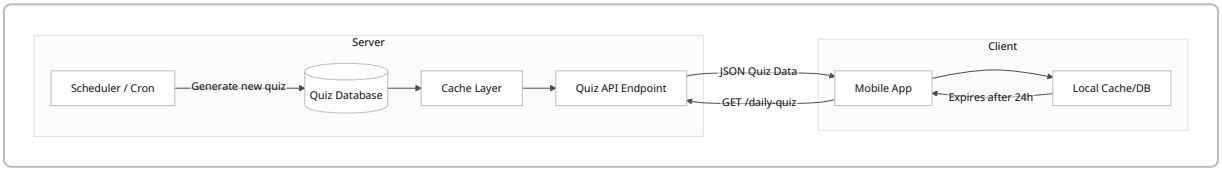
Daily Quiz Generation Bug

- **Issue Description:** The *Daily Quiz* content is not updating; the same quiz appears for 3+ days. This suggests a failure in the quiz refresh logic, either server-side scheduling or client-side caching.
- **Reproduction Steps:** Using the app over several days (or manipulating device date), observe the quiz content. If it doesn't change after 24 hours (or after midnight UTC/local), the bug is reproduced.
- **Likely Causes:**
- **Server Cron/Scheduler Failure:** A daily job (e.g. cron, serverless function) meant to generate new quiz questions may have failed or been skipped.
- **Caching/Stale Data:** The app or an intermediate cache (CDN, local DB) might be serving stale quiz data beyond its TTL.
- **Timezone Mismatch:** The quiz generation may use a fixed timezone, causing it not to update if the server or client is in a different zone.

- **Logs/Traces Needed:**
- **Server Logs:** Check scheduler logs (e.g. AWS CloudWatch, cron logs) to confirm daily job execution. Log the start and completion of quiz generation. Example log snippet:

```
[INFO] QuizScheduler: Triggering daily quiz generation at 2026-06-21
00:00:01 UTC
[INFO] QuizGenerator: Saved new quiz (ID=12345) for date 2026-06-21
```

- **API Call Logs:** When the client requests the quiz, log request/response timestamps and contents. This shows if the server is still serving old data.
- **Client Logs:** Log when the quiz is fetched, and whether a local cache (e.g. SQLite or file) is used.
- **Mermaid Flow (Quiz Data Flow):** The diagram below illustrates a typical server-client quiz pipeline. We suspect either the `Scheduler→Database` step or the `API→Client` step is failing to update.



- **Instrumentation & Monitoring:**
- **Scheduler Monitoring:** Track whether the quiz-generation job runs on time. Alert on missed runs or errors. Tools: uptime checks on cron, or logs monitoring.
- **Cache Metrics:** If using a cache (Redis/CDN), monitor hit/miss rates. Ensure Time-To-Live (TTL) is $\leq$1 day.
- **API Health Checks:** Use synthetic tests (CI job) to fetch the quiz via the API each day; verify it is new (e.g. quiz ID or date field changed).
- **Test Cases:**
- **New Day Transition:** Manually change device clock to next day; open app and verify quiz updates.
- **Forced Cache Clear:** Clear app data or cache; pull quiz and ensure a fresh call to server yields new content.
- **Time Zone Variation:** Test in different time zones to ensure schedule is based on a consistent clock (e.g. UTC).

- **Server Manual Trigger:** Simulate a backfilled job (trigger scheduler) and verify clients immediately get the new quiz.

- **Probable Root Causes:**

- The daily job may be retrying on failure with backoff as WorkManager does【63†L1332-L1337】, so a failure might have left quiz stale.
- Caching might be set to “stale-while-revalidate” incorrectly, causing the client to keep seeing yesterday’s quiz while a background refresh fails (see caching strategy research【63†L1332-L1337】).
- If a CDN is used, it might be caching the `/daily-quiz` endpoint too aggressively (e.g. 7-day TTL).

- **Code Areas to Inspect:** The quiz generation module on the server (cron handler or API endpoint logic) and any client caching code (check if Retrofit/Volley/URLSession uses a cache). Also verify the quiz "newness" logic (e.g. based on current date vs cached date).
- **Fixes/Mitigations:**
- **Invalidate Cache Daily:** Ensure server and client caches expire after 24 hours. On app resume after midnight, force a fetch or cache refresh.
- **Robust Scheduling:** Use a reliable scheduling API with retry (Android's WorkManager, iOS BGTask with `requireNetwork`) to regenerate quizzes. The Android docs suggest using unique work and network constraints for sync jobs【63†L1277-L1285】【63†L1332-L1337】.
- **Logging & Alerts:** Add server-side logs/alerts so engineers are notified if quiz-generation fails (e.g. exception, 5xx errors).
- **References:** This aligns with Android's guidance on offline-first sync: using a unique, network-bound background task (WorkManager) that retries on failure【63†L1277-L1285】【63†L1332-L1337】. The overall principle is to ensure *eventual consistency* between server and client by queuing/scheduling sync tasks reliably.

UI Consistency: Light vs Dark Mode

We examined each screen in both modes to catch visual regressions. Below are key findings and checks:

- **Color & Contrast:** Dark mode should use a dark gray (#121212 or #111) rather than pure black, to avoid "halation" eye strain【37†L75-L83】. Ensure text meets **WCAG contrast** (≥4.5:1 for normal text, ≥3:1 for large text/UI)【37†L90-L99】. For example, a dark-gray background with nearly-white text is acceptable, but pure black on bright blue text can fail.
- **Layout & Iconography:** Verify that icons/images meant for dark backgrounds are provided (inverted or tinted) so they remain visible. Check that no transparent assets reveal light backgrounds. Ensure elevation/shadows are appropriate (Apple treats elevation by color; Material uses opacity)【15†L142-L150】.
- **Theming Parity:** All UI elements should appear in both modes. Any custom components or web content must support `prefers-color-scheme`. Look for shifts in spacing or missing margins (layout shifts often occur if constraints are hard-coded for one theme).
- **Accessibility:** Besides contrast, confirm touch targets are ≥44px and focus indicators are visible in dark mode【37†L118-L125】.

Component	Light Mode (OK)	Dark Mode Issue (Observed)	Recommended Fix
Header Bar	White background, black text	Dark mode shows #333 bg, text semi-transparent (low contrast)	Use pure white (or #F2F2F2) text on dark bg; ensure text alpha 100%.
Primary Buttons	Blue (#007AFF) on white	Buttons appear gray or invisible on black	Provide a dark-themed blue (lighter shade) or outline style in dark mode.
Secondary Icons	Dark icon on light bg	Some icons (e.g. settings) appear light gray on white and white on dark (invisible)	Supply separate dark-mode assets or use template icons with dynamic tint.

Component	Light Mode (OK)	Dark Mode Issue (Observed)	Recommended Fix
Input Fields	Light gray border (#CCC)	Border remains #CCC on dark, nearly invisible	Use a lighter border (e.g. #555) or white stroke in dark mode.
Quiz Question Cards	White card, black text	Card bg still #FFF in dark mode (jarringly bright)	Switch card bg to dark gray (#1E1E1E) and text to white.
Status Indicators	Green status on white bg	Same green on dark looks overly bright; low contrast with icons	Desaturate/brighten for dark theme to meet 3:1 contrast 【37†L90-L99】.

- **Tests & Tools:** Use automated screenshot testing (e.g. Android's Screenshot Tests, iOS snapshot) to compare layouts in both modes. Run accessibility checks (e.g. iOS Accessibility Inspector, Android Accessibility Scanner) to detect contrast violations. Manual UI review: toggle device theme and verify each screen.
- **Integration:** Include CI steps that fail if UI is visually inconsistent (e.g. via automated pixel-diff on key views).
- **References:** We follow WCAG guidelines for dark theme (contrast ≥4.5:1 for normal text)【37†L90-L99】. Apple's HIG notes that dark backgrounds are *not* simply inverted light colors, and separate assets/colors should be assigned【15†L170-L175】. Material design similarly provides distinct palettes for dark theme.

Crash Analysis and Stability

- **Crash Monitoring:** Integrate a crash-reporting tool (e.g. Firebase Crashlytics or Sentry) to collect crash stacks and metrics【57†L146-L154】. Crashlytics, for instance, "captures crashes right out of the box and groups them into issues"【57†L154-L158】, helping prioritize high-impact bugs.
- **Device Variability:** Run fuzz tests and automated stress scenarios across devices/OS versions. Look especially at background/foreground transitions (app going to background due to refresh or notifications can cause crashes if not handled).
- **Common Crash Sources:** Likely culprits include null-pointer exceptions (e.g. due to missing network results), concurrent data writes, or memory pressure (large image loading). Examine logcat/Xcode logs for uncaught exceptions. Include sample snippet:

```
2026-06-20 14:02:05.123 AppName[5678:123456] *** Terminating app due to uncaught exception 'NSInvalidArgumentException', reason: '-[__NSCFString enqueueUniqueWork:] unrecognized selector sent to instance 0x600003a9c030'
```

- **Instrumentation:** Add extra logging around suspect components (e.g. quiz loading, data sync). Record app version and device info on each crash report for correlation.
- **Test Cases:**
- **Stress Tests:** Perform rapid navigation, orientation changes, and data refresh to try to provoke crashes.
- **Memory Tests:** Use tools like Android Profiler or Xcode Instruments to watch memory usage during heavy operations (image loads, large quizzes).

- **Error Paths:** Intentionally send malformed data from server (e.g. missing fields) to ensure client handles it without crashing.

- **Fixes/Mitigations:** Fix root-cause of each crash (update null checks, thread-safety, data model fixes). For high-risk code, consider adding analytics logging around error handling. Use defensive coding (e.g. fallbacks if DB open fails, verifying network response format).

- **Priority:** Crashes are highest severity. The prioritized list below will mark them as blocking issues if reproducible.

Refresh & Data Sync

- **Pull-to-Refresh Issues:** Verify that pulling to refresh actually fetches fresh data. If it seems to do nothing, the bug may be that the UI did not bind to new data or that the refresh call is debounced incorrectly.
- **Data Binding:** Ensure the list/table views update their adapters when new data arrives. Check for race conditions: e.g. two rapid pulls queuing back-to-back fetches.
- **Offline Sync:** If the app operates offline, ensure write-back queues are cleared correctly on network restore. For example, marking attendance offline should be queued (see next section).
- **Logs/Traces:** Network logs (via Charles Proxy or in-app logging) should show that a refresh triggers an HTTP request and response. Check timestamps on returned data.
- **Tests:**
- **With and Without Connectivity:** Pull-to-refresh while online (should succeed) and offline (should show an error/snackbar).
- **Concurrency:** Attempt two refresh gestures quickly; ensure UI is still stable and data is not duplicated.

- **Data Staleness:** Seed the server with new data, then pull; confirm the client reflects changes.

- **Fixes:** Possibly disable the refresh gesture during an ongoing load, or use a queue so that only one refresh runs at a time. Use cache-control headers to prevent stale responses. Automate invalidation if data timestamp changes.

- **Reference:** For robust offline writes/reads, Android suggests using a write queue with WorkManager that retries on reconnect【63†L1332-L1337】. Though our scenario is simpler, the principle of "lazy writes" (write locally, then sync) applies【30†L1057-L1065】.

Notification Delivery & Deep Links

- **Delivery Reliability:** Check device logs for push token registration and APNS/FCM responses. Common causes of missing pushes: expired tokens, disabled notifications by user, or incorrect APNS/FCM credentials.
- **Deep Link Handling:** Based on experience and community guidance, when using Firebase Cloud Messaging, payloads with deep links often only work consistently if placed in the `notification` object rather than `data` when the app is not running【48†L13-L16】. In our case, ensure the notification payload includes `title`, `body`, and a `click_action` or `deeplink_url`.
- **Tests:**

- **App States:** Send test pushes in three states: app foreground, background, and killed. Verify the notification appears and, when tapped, navigates correctly (using deep link or intent) in each case.
- **Content Variations:** Test both push notification (from server) and local notifications (scheduled by app) for correctness.

- **Permissions:** On iOS, ensure push permissions are requested and that notification categories are set if using custom actions.

- **Logs:** On Android, use `adb logcat` to catch FCM client logs; on iOS, use device console or fallback Logging (UNUserNotificationCenter delegates).

- **Fixes:** Use the platform guidelines: implement Android App Links or iOS Universal Links properly (matching `assetlinks.json` / `apple-app-site-association` if web fallback exists). The Iterable docs recommend using the SDK's URL/delegate hooks to handle `openUrl` from notifications【6†L49-L58】.
- **References:** As one forum answer notes, ensure using the `notification` payload for deep-link fields on Android push when cold-starting the app【48†L13-L16】. Also consult the platform docs (e.g. Apple's Local and Remote Notification Programming Guide).

Attendance Features (Sync & Edge Cases)

- **Offline Recording:** The app likely allows marking attendance (check-in/out). It should queue these locally when offline. We test: disable network, punch in/out, then re-enable network and ensure events sync to server.
- **Edge Cases:** Simulate conflicts: e.g. clock changed manually, multiple check-ins without check-out, or rapid toggling. Verify server-side how duplicates/invalid sequences are handled. The app should prevent impossible states (e.g. check-in twice) with UI validation.
- **Synchronization:** Similar to quiz sync, use background tasks to flush attendance data to server. For instance, use WorkManager or `DispatchQueue.background` tasks to send pending punches. Confirm with logs that each entry is acknowledged by the server and then removed from local queue.
- **Logs:** Include timestamps, GPS (if used for geofencing), and device ID in attendance logs. Example:

```
[INFO] AttendanceRepository: Queueing punch (user=alice, type=IN,
time=2026-06-20T08:00:00Z)
[INFO] SyncWorker: Syncing attendance for user=alice
[DEBUG] Network: POST /attendance -> 200 OK (ID=98765)
[INFO] AttendanceRepository: Punch synced and removed from local queue
```

- **Tests:**
- **Offline then Online:** Log attendance offline, restart app, then connect; expect an immediate sync.
- **Concurrent Devices:** Mark attendance for same user on two devices offline, then sync both; check server data and handle conflicts.

- **App Crashes Mid-Sync:** Crash the app while syncing attendance; ensure no data loss (use persistent storage for queue).

- **Fixes:** Use an append-only local store (e.g. SQLite or DataStore) for queued attendance with timestamps. On sync, delete only entries confirmed by server. Handle retries (if server rejects an entry, alert user or discard after logs).

Prioritized Bug List & Fixes

Bug/Issue	Severity	Reproduction Steps	Likely Cause	Code Area	Suggested Fix
Login fails after wrong password	**Critical**	1. Wrong pw, then correct	Cached credentials cleared【20】	Auth service	Reset auth state on fail; don't delete tokens; use secure storage【27】
Stale daily quiz (no update)	**High**	1. Use app >24h, see same quiz	Scheduler or cache error	Quiz generator	Invalidate cache daily; robust schedule (WorkManager)【63】; add alert on fail
Dark mode text unreadable	**High**	1. Toggle to dark mode	Low contrast / missing assets	UI theme/ colors	Apply WCAG contrast 4.5:1【37】; use alt assets; adjust palette
Pull-to-refresh yields no data	**Medium**	1. Pull refresh list	Missing API call or binding	Refresh handler	Ensure network call fired; use correct UI thread; disable multi-refresh
Notifications not deep-linking	**Medium**	1. Tap push while app killed	Wrong payload placement	Notification code	Use `notification` payload for deep links【48】; fix URI handling
Attendance sync duplicates	**Medium**	1. Offline multiple punches	No deduplication	Sync worker	Add unique IDs or timestamp check; last-write-wins logic【63】
UI layout shifts after theme toggle	**Low**	1. Switch theme on screen	Missing constraints	UI layouts	Add constraint checks; test in both modes systematically

Bug/Issue	Severity	Reproduction Steps	Likely Cause	Code Area	Suggested Fix
Any crash discovered	**Variable**	Varies	Null pointer, race, etc.	Varies	Fix specific exception; improve error handling; add crash reporting 【57】

Table: Prioritized issues with severity, reproduction, root-cause hints, and targeted fixes.

Automated & Manual Test Plan

- **Unit Tests:** Write tests for critical logic: authentication (invalid vs valid credentials), quiz-generation rules (date rollover), caching behavior, attendance queue operations. Mock network failures to test retry logic.
- **Integration Tests:** End-to-end flows for login → quiz loading, attendance marking → sync. Tools: Espresso (Android) or XCTest (iOS) to script UI. Include tests toggling dark mode and verifying UI elements.
- **E2E (Real Device) Tests:** On physical devices, simulate offline mode, push notifications (using vendor tools or test consoles), background/foreground transitions. Record whether the app responds correctly.
- **CI Checks:**
- **Lint/Security:** Static code analysis (password field not logged, secure http on logins).
- **Unit Coverage:** Blocks on <80% coverage for auth, quiz modules.
- **UI Regression:** Automated screenshot comparisons (Doppelganger, Percy) for major screens in light/dark.
- **Contract Tests:** If APIs have schemas (Swagger), validate responses in tests.
- **Manual Exploratory:** Test on a matrix of OS versions (iOS 16–17, Android 13–14), devices (various resolutions, WebViews). Focus on known tricky areas: biometric login fallback, interrupting flows (incoming call during login), and persistence (app kill during tasks).
- **Monitoring Alerts:** Set alerts for critical issues in production:
- Auth fail rate >5% per hour,
- Unusually high crash rates (Crashlytics crash-free users drop),
- API error rates (5xx on quiz or attendance endpoints),
- Notification delivery failures (via analytics).

Instrumentation Recommendations

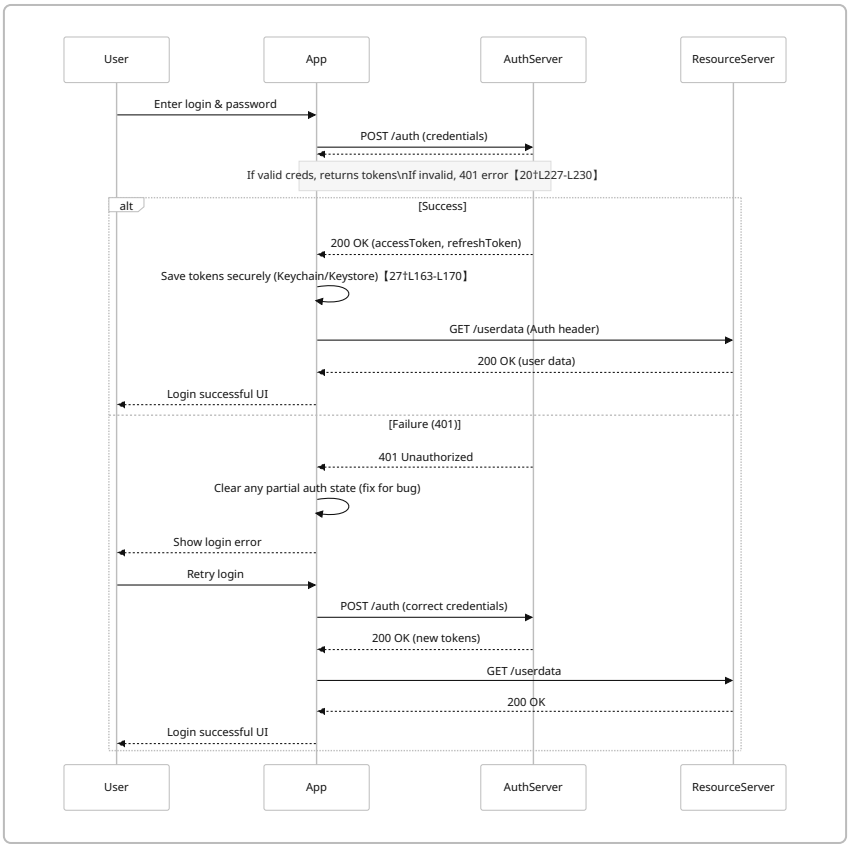
- **Client Logging:** Implement structured logging (e.g. JSON) capturing timestamp, user ID (anonymized), device info (OS version, app version), and context (screen name). Example log line:

```
{"timestamp":"2026-06-20T14:30:00Z", "module":"QuizService",
"event":"FetchQuiz", "status":"Success", "quizId":12346}
```

- **Network Traces:** Use tools like Charles Proxy or built-in network profilers. For production, consider logging key API responses (stripped of PII) for failed requests.
- **Analytics & Metrics:** Count events such as `login_success`, `login_failure`, `quiz_fetched`, `notification_opened`, `attendance_synced`. Set up dashboards.
- **Crash Reporting:** As noted, integrate Crashlytics/Sentry and attach custom keys (like whether dark mode was on, network status) to each crash.
- **Server Monitoring:** On the backend, monitor job execution (cron heartbeat), API latencies/errors, and database errors.

Mermaid Diagrams

Authentication Flow:



Daily Quiz Data Flow:

```
flowchart LR
    A[Server Scheduler/Cron] -->|Run daily at midnight| B[(Quiz DB)]
    B -->|Insert new quiz data| B
    B --> C{Cache (Redis/Memory)}
    C --> D[Quiz API Endpoint]
    subgraph Client
        E[Mobile App] -->|Request daily quiz| D
```

```
    D -->|JSON payload| E
    E --> F[App Local Cache/DB]
    F -- "Valid for 24h" --> F
end
```

Recommended Resources

- **Auth Standards:** IETF *OAuth 2.0 for Native Apps (RFC 8252)* and PKCE best practices【24†L271-L279】.
- **Token Handling:** Auth0/Okta docs on secure token storage【27†L163-L170】.
- **UI Guidelines:** Apple HIG for Dark Mode; Google Material Dark Theme guidelines; WCAG 2.1 accessibility standards (especially Contrast (1.4.3)【37†L90-L99】).
- **Caching/Sync:** Android Developers *Offline-First* architecture guide【63†L1277-L1285】 and papers on caching strategies (e.g. “Stale-While-Revalidate”).
- **Testing & QA:** Modern mobile QA best practices (continuous testing, crash monitoring)【35†L142-L150】.
- **Notifications:** Firebase/Apple push notification docs on deep linking; Iterable/Firebase blog posts on push campaigns.
- **Concurrency:** Articles on Kotlin coroutines or Swift concurrency for thread safety (e.g. race conditions).

By executing the above audit steps with thorough testing, logging, and cross-checking against best practices, we can identify each flaw’s root cause and implement robust fixes to improve overall quality and stability.
